PROJEKTI IZ ELEKTRONIKE

dr. David Rihtaršič

2021-November

Kazalo

1 NAČRTOVANJEM ELEKTRONSKIH VEZIJ **1**

1.1 Stikalne sheme 1

1.1.1 NALOGA: Stikalne sheme 4

1.2 Tiskana vezja 4

1.2.1 NALOGA: Risanje TIV 4

1.3 Virtualna vezja 5

1.3.1 NALOGA: Sestavljanje virtualnih vezij 5

2 SVETLOBNI SPOJNIKI **7**

2.1 Vklop porabnika s tranzistorjem 7

2.1.1 NALOGA: Vklop žarnice s tranzistorjem 7

2.2 Komparator napetosti 7

2.2.1 NALOGA: Komparator napetosti 7

2.3 Schmittov sprožilnik 8

2.3.1 NALOGA: Komparator napetosti s šmitovim sprožilnikom 8

3 UPORABA MIKROKRMILNIKOV **9**

3.0.1 NALOGA: Glavni deli krmilnika 10

3.0.2 NALOGA: Osnovne nastavitve in testni program 10

3.1 Programiranje krmilnikov Arduino 10

3.1.1 NALOGA: AVRDUDE - program za prenos strojne kode 11

3.2 Uporaba vhodno-izhodnih priključkov na kmilniku Arduino 11

3.2.1 NALOGA: Shema krmilnika Arduino NANO 11

3.3 Vhodno-izhodne enote mikrokrmilnika 11

3.3.1 NALOGA: Krmilni registri mikrokrmilnika 12

3.4 Branje napetostnega potenciala 13

3.4.1 NALOGA: Branje napetostnega potenciala 13

3.4.2 NALOGA: Uporaba upora proti napajanju 13

3.5 Prehodni stiki stikal 14

3.5.1 NALOGA: Večkratni preklopi 14

4 KOMUNIKACIJSKI VMESNIKI 17
4.1 Serijska komunikacija UART . . . 17
4.1.1 NALOGA: Osnovni parametri UART protokola . . . 17
4.1.2 NALOGA: Uporaba serijske UART komunikacije . . . 18
4.1.3 NALOGA: Časovni potek napetosti UART komunikacije . . . 18
4.2 I2C komunikacija . . . 18
4.2.1 NALOGA: Priključitev I2C LCD-ja . . . 20
4.2.2 NALOGA: Izpis na LCD . . . 20

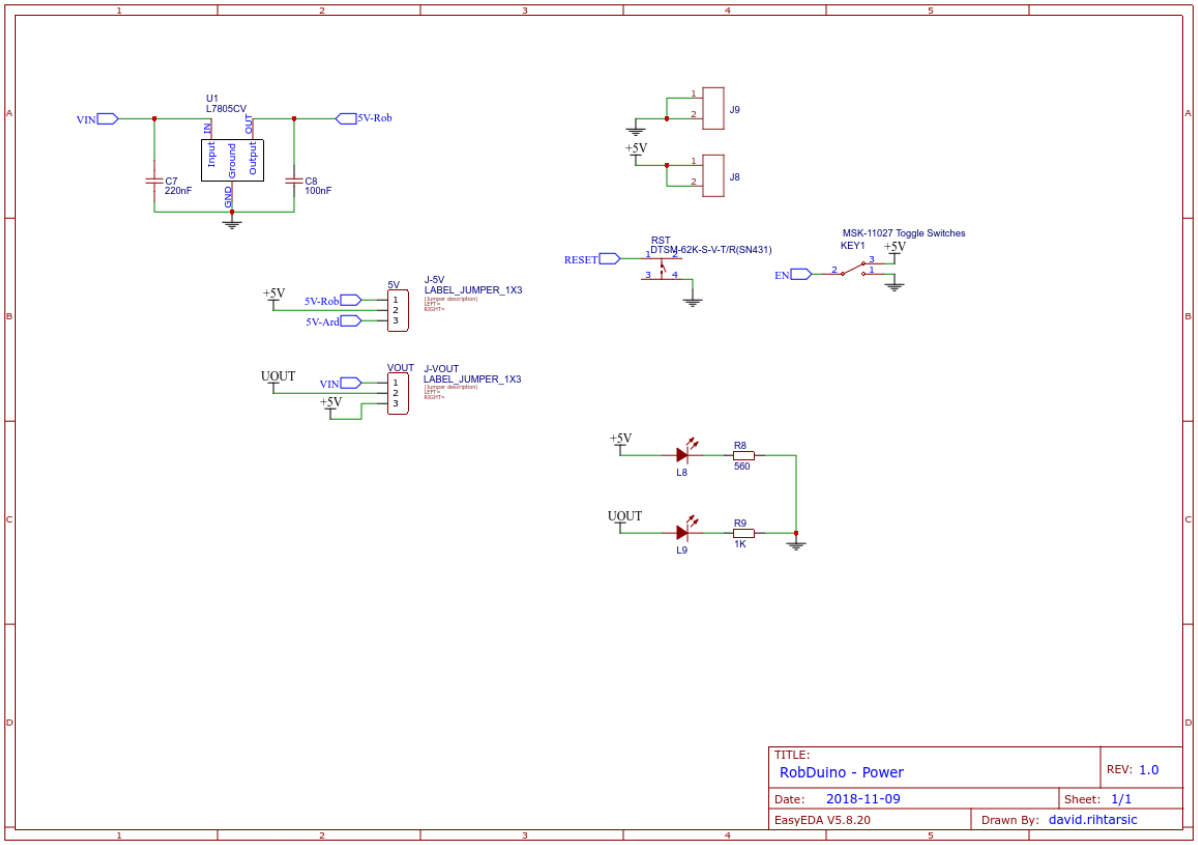
5 MODULACIJA 21
5.1 Amplitudna modulacija . . . 21
5.1.1 NALOGA: Pošiljanje IR signala . . . 21
5.2 Frekvenčna modulacija . . . 21
5.2.1 NALOGA: Blinker . . . 21
5.3 Pulzno-širinska modulacija . . . 21
5.3.1 NALOGA: Krmiljenje servo-motorja . . . 21

1 NAČRTOVANJEM ELEKTRONSKIH VEZIJ

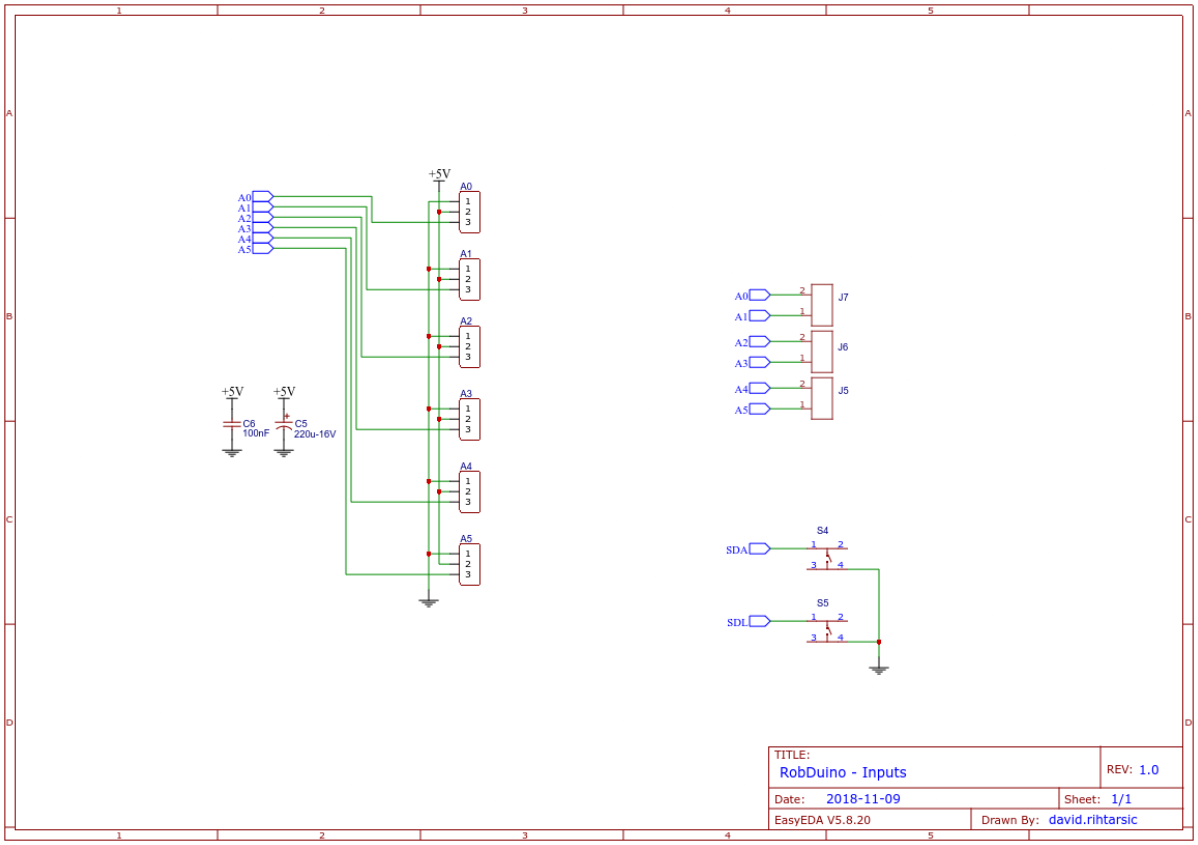
Tudi pri pedagoškem procesu je pomembno, da so vezja narisana nazorno (tako sheme, kot tudi sestavljanje vezja na prototipni ploščici). V ta namen lahko uporabljate različna orodja. Omenili bomo vsaj dva, ki sta prosto-dostopna in bi priporočali njihovo uporabo.

1.1 Stikalne sheme

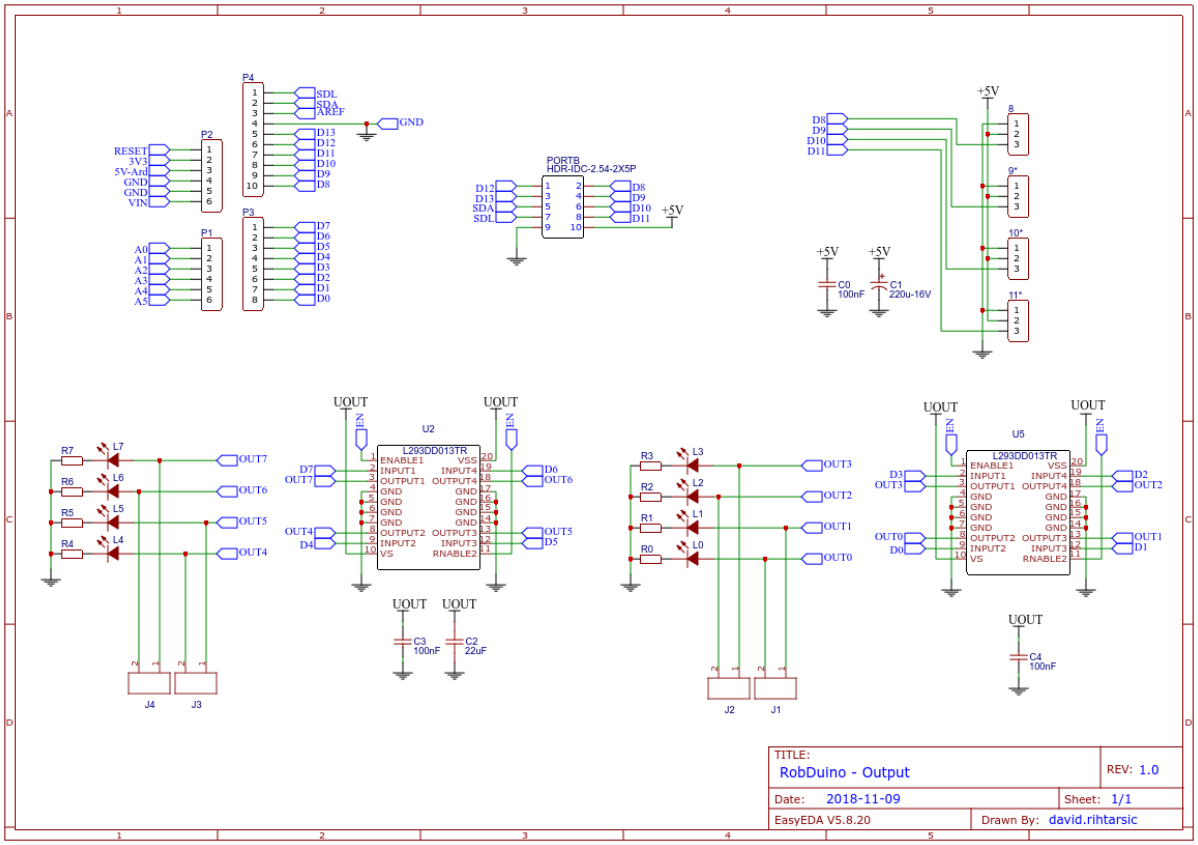
EasyEDA je spletno orodje, ki je namenjeno risanju elektronskih vezij, načrtovanju TIV in izdelavi potrebnih datotek za njihovo izdelavo. Primer stikalne sheme krmilnika RobDuino lahko vidite na sl. 1.1,sl. 1.2 in sl. 1.3.



Slika 1.1: Stikalna shema modula RobDuino - napajalni del.



Slika 1.2: Stikalna shema modula RobDuino - vhodni del.



Slika 1.3: Stikalna shema modula RobDuino - izhodni del..

1.1.1 NALOGA: Stikalne sheme

V programskem orodju EasyEDA narišite shemo astabilnegamultivibratorja, izvozite sliko sheme in jo vključite v poročilo.

1.2 Tiskana vezja

1.2.1 NALOGA: Risanje TIV

Za to vezje izrišite TIV in izpišite seznam elektronskih komponent. Izgled TIV izvozite in vstavite v poročilo. Prav tako vstavite seznam komponent.

1.3 Virtualna vezja

Sestavljanje vezij pa na prototipni plošči pa je drugačen proces in za začetnike precej zahteven, zato je priporočljivo, da uporabljate orodja kot so TinkerCAD-Circuits.

> ### 1.3.1 NALOGA: Sestavljanje virtualnih vezij
>
> V programskem orodju TinkerCAD-Circuits sestavite vezje na prototipni ploščici in sliko vstavite v poročilo.

2 SVETLOBNI SPOJNIKI

Pred davnimi časi, ko svet še ni slišal za digitalno tehnologijo, se je gospod Samuel Morse domislil, da bi črke kodiral v kratke in dolge pulze. Te pa bi lahko kar najlažje pošiljal od ene točke do druge na najrazličnejše načine … in telekomunikacije so prijokale na svet.

2.1 Vklop porabnika s tranzistorjem

Pogosto moramo porabnike skozi katere tečejo večji tokovi ($I > 500\,mA$) vključiti s tranzistorjem.

> ### 2.1.1 NALOGA: Vklop žarnice s tranzistorjem
>
> 1. Za pošiljanje Morsejevih znakov uporabite žarnico. Ali nek drug vir z večjo svetilnostjo.
> 2. Dolge in kratke pulze bomo pošiljali s svetlobnim oddajnikom. Načrtujete ustrezno rešitev (narišite shemo vezja) tako, da bomo s pritiski na tipko vklapljali in izklapljali svetilo (uporabite žarnico [12V in 0,6 A]).
> 3. Bodite pozorni, na električne omejitve tipke, ki jih najdete v navodilih za uporabo tipke. Načrtujte ustrezno rešitev.

2.2 Komparator napetosti

V kolikor želimo ločiti med dvema napetostnima nivojema, lahko uporabimo komparator napetosti s primerno izbrano (ali celo nastavljivo) referenčno vrednostjo.

> ### 2.2.1 NALOGA: Komparator napetosti
>
> 1. Izdelajte svetlobni sprejemnik, v katerega boste za zaznavanje osvetljenosti uporabili elektronski element s hitrim odzivom.

2. Analogni signal senzorja modificirajte tako, da boste lahko nedvoumno podajali informacijo (npr. LED svetilo), ki jo je poslal svetlobni oddajnik.
3. Z osciloskopom zajemite časovni potek napetostnega potenciala na vhodnem in na izhodnem priključku komparatorja.

2.3 Schmittov sprožilnik

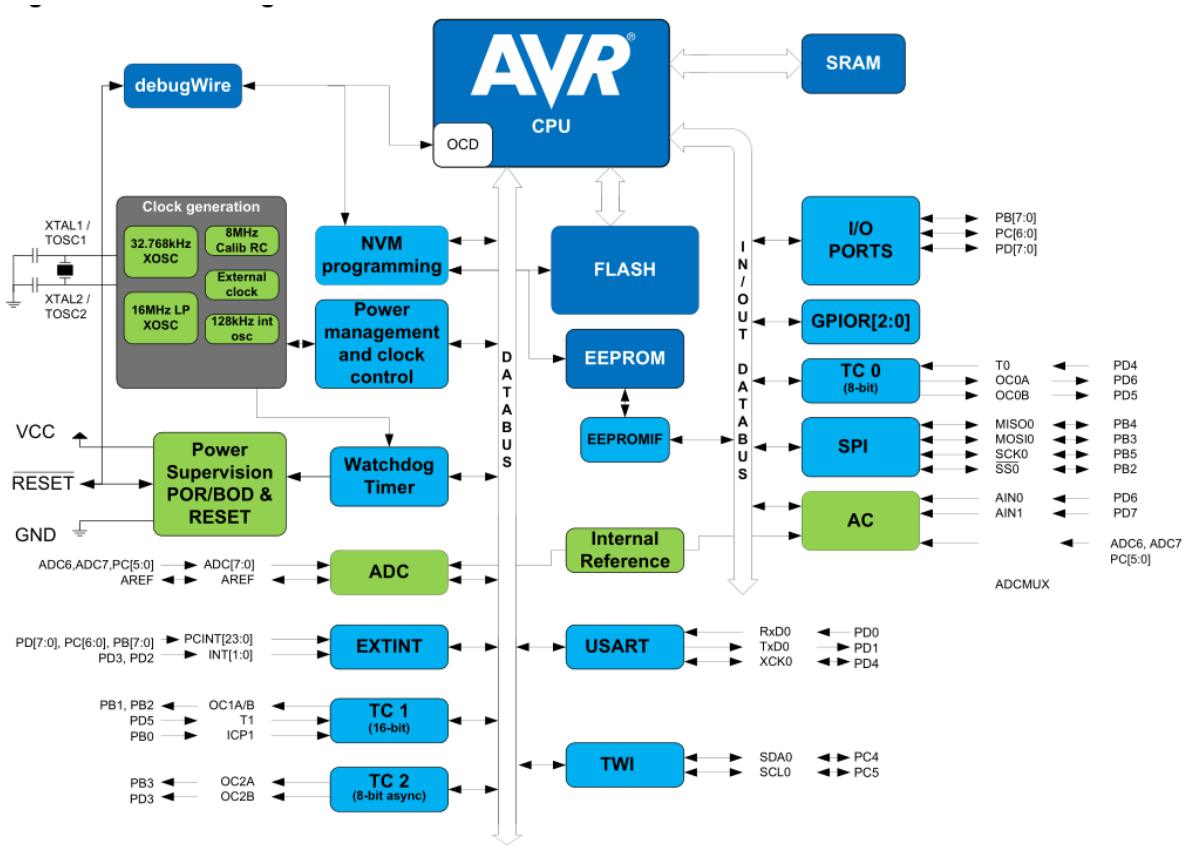
Iz prejšnje naloge lahko ugotovimo, da je izhodni signal svetlobnega senzorja opremljen z neželeno motnjo. Če v tem primeru želimo ločiti dve različni stanji senzorja, komparator napetosti ni dobra rešitev. Ob prehodu senzorja iz enega stanja v drugega tako dobimo na izhodu komparatorja več prehodov, čeprav se je osvetljenost senzorja spremenila le enkrat. Tako lahko ugotovimo potrebo po histerezi s katero bomo bomo mejno vrednost razmejili na dve mejni vrednosti. Tako ločevanje nam omogoča schmittov sprožilnik.

2.3.1 NALOGA: Komparator napetosti s šmitovim sprožilnikom

1. Iz grafa prejšnje naloge, ki predstavlja časovno odvisnost napetostnega potenciala vhodnega in izhodnega signala, odčitajte nove mejne vrednosti ($U_{h1}\ in\ U_{h2}$).
2. Komparator napetosti iz prejšnje naloge zamenjajte s komparatorjem napetosti s šmitovim sprožilnikom (narišite stikalno shemo).
3. Z osciloskopom zajemite časovni potek napetostnega potenciala na vhodnem in na izhodnem priključku komparatorja napetosti s šmitovim sprožilnikom.

3 UPORABA MIKROKRMILNIKOV

Za projekte, ki vključujejo programabilno elektroniko, pogosto uporabljamo že izdelane krmilnike iz družine Arduino. Na teh vezjih lahko najdemo mikrokrmilnike proizvajalca Atmel. Najbolj pogosto uporabljena krmilnika (Arduino Uno in Arduino NANO) temeljita na mikrikrmilniku Atmega328p. Blokovna shema tega mikrokrmilnika je prikazana na sl. 3.1.



Slika 3.1: Blokovna shema mikrokrmilnika ATmega238.

3.0.1 NALOGA: Glavni deli krmilnika

1. Na blokovni shemi označi pomembne dele krmilnika in zapiši njihov glavni namen (funkcijo). Glavni sestavni deli so: generator delovnega takta (ura), centralno procesna enota, začasni (delovni) spomin, trajni spomin, vhodno-izhodne enote, komunikacijske enote …

Mikrokrmilniki na krmilnikih Arduino so že opremljeni s programom (angl. »boot loader«), ki poskrbi za ustrezno prepisovanje programske vsebine, ki jo računalnik pošlje preko USB vodila. Tako lahko enostavno programiramo mikrokrmilnike, ki so na ploščah krmilnikov Arduino.

3.0.2 NALOGA: Osnovne nastavitve in testni program

1. Iz programskega okolja Arduino IDE prepišite nastavitve programatorja ter izpolni tbl. 3.1:

Table 3.1: Nastavitveni parametri programatorja.

Nastavitveni parameter	Vrednost nastavitvenega parametra
Board (Plošča)	
~~Processor~~ (Mikrokrmilnik)	
Port (vrata)	
Programmer (programator)	

2. Iz primerov, ki so vključeni v programskem okolju Arduino IDE izberite Blink.ino in ga preskusite.

3.1 Programiranje krmilnikov Arduino

Za programiranje mikrokrmilnika skrbi odprtokodna programska koda - avrdude, ki se izvaja v ozadju programskega okolja Arduino IDE. Proces programiranja lahko bolj natančno spremljamo tako, da vključimo

```
File -> Preferences:

Shov verbose output during: [x] compilation [x] upload
```

3.1.1 NALOGA: AVRDUDE - program za prenos strojne kode

1. Prepišite ukazno vrstico programa avrdude za premos strojne kode in
2. opišite pomen parametrov.

Več o parametrih lahko najdete na spletni strani avrdude

3.2 Uporaba vhodno-izhodnih priključkov na kmilniku Arduino

3.2.1 NALOGA: Shema krmilnika Arduino NANO

1. Oglejte si shemo krmilnika Arduino NANO in
2. skušaj ugotoviti na kateri priključek IO enote je priključena LED, ki je na krmilniku.
3. Izpolni tbl. 3.2

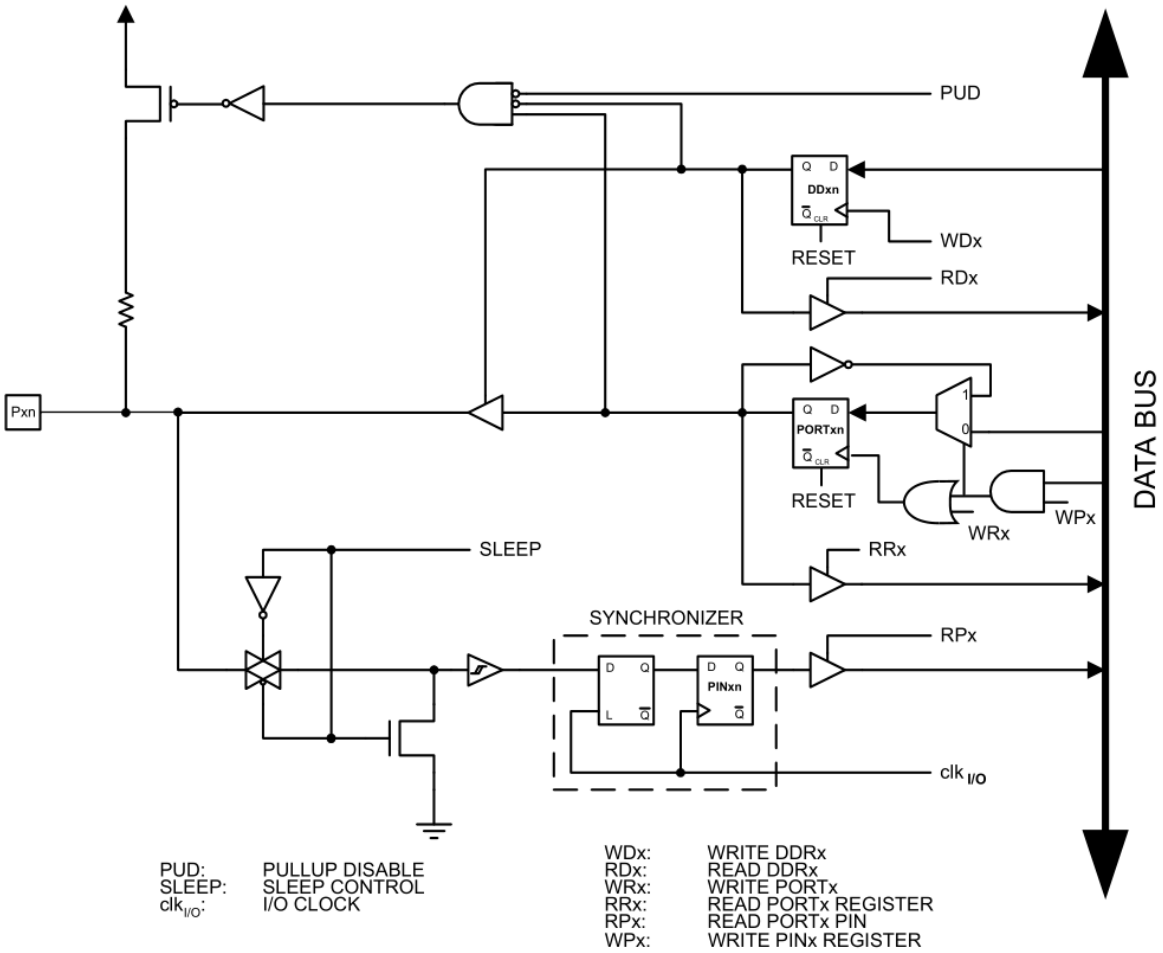
Table 3.2: Razporeditev priklučkov na krmilniku in mikrikrmilniku.

Funkcija krmilnika	Arduino NANO[a]	ATmega238[b]
LED na plošči		
TxD		
RxD		

[a] Oznaka priključka na krmilniku Arduino NANO.
[b] Oznaka priključka na mikrokrmilniku ATmega238.

3.3 Vhodno-izhodne enote mikrokrmilnika

Vhodno-izhodne enote mikrokrmilnika so dvo-smerne z možnostjo nastavitve upora proti napajanju. Stikalna shema enega priključka na neki vhodno-izhodni enoti je prikazana na sl. 3.2.



Slika 3.2: Shema priključka n na vhodno-izhodni enoti x.

Čeprav je shema na sl. 3.2 nekoliko bolj kompleksna, lahko ugotovimo, da za nastavitev delovanja vhodno-izhodne enote potrebujemo le dva signala:

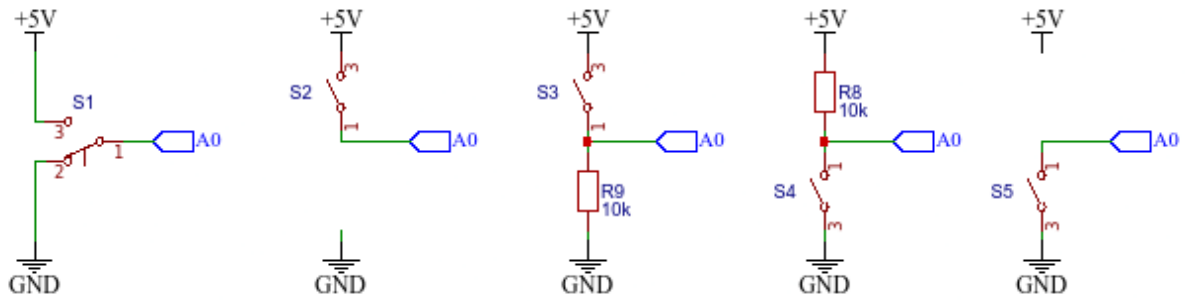
- WDx - (DDRx) - za določanje vhodne oz. izhodne funkcije priključka
- WPx - (PORTx) - za določanje vrednosti logičnega stanja na priključku x.

3.3.1 NALOGA: Krmilni registri mikrokrmilnika

1. Preoblikuj program Blink.ino tako, da boš krmilil vklop in izklop LED z nastavitvijo krmilnikh registrov.
2. Predstavite programsko kodo.

3.4 Branje napetostnega potenciala

Branje napetostnega potenciala je najbolj pogosto uporabljeno pri vzorčenju pritiska tipke. Poglejmo si tak primer.



Slika 3.3: Različne možnosti priključitve tipke na digitalni vhod.

3.4.1 NALOGA: Branje napetostnega potenciala

1. Sestavite prve tri različice priključitve tipke na digitalni vhod krmilnika A0 in preskusite spodnji program.
2. Razložite (ne-)delovanje.

```
void setup(){
    pinMode(13, OUTPUT);
    pinMode(A0, INPUT);
}

void loop(){
    if ( digitalRead(A0) == HIGH)
        digitalWrite(13, HIGH);
    else
        digitalWrite(13, LOW);
}
```

3.4.2 NALOGA: Uporaba upora proti napajanju

1. Spremenite delilnih napetosti tipka - upor tako, da bo upor vezan proti napajanju in nato spremenite program tako, da bo delovanje ostalo enako prejšnji nalogi. Priložite stikalno

shemo vezja in preoblikovan program.
2. Odstranite upor in preverite delovanje vezja. Kaj opazite, razložite delovanje.
3. Vključite notranji upor proti napajanju, ki se nahaja v mikrokrmilniku ob vsakem vhodno-izhodnem priključku (glej sl. 3.2). Priložite programsko spremembo.

3.5 Prehodni stiki stikal

Fizični preklopni elementi (kot je tipka) imajo tudi fizikalne lastnosti kot so trdota, trdnost, prožnost, elastičnost… Zaradi vseh teh lastnosti je preklop tipke iz enega položaja v drugega lahko tudi nekoliko nepredvidljiv.

3.5.1 NALOGA: Večkratni preklopi

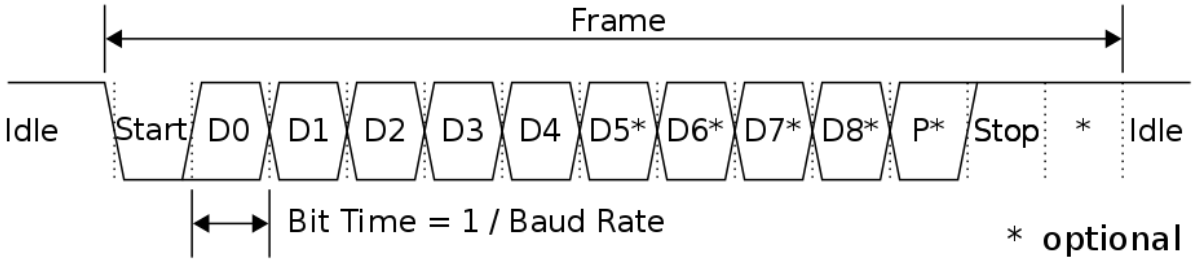
1. Preskusite spodnji program in opišite njegovo delovanje.
2. Opišite zaznane težave.
3. Z osciloskopom ujemite enega od prehodov (0->1 ali 1->0) in
4. problem rešite:
   - programsko ter
   - elektronsko. Obe rešitvi dokumentirajte.

```
void setup()
{
    const int LED = 13;
    pinMode(LED, OUTPUT);
    pinMode(A0, INPUT_PULLUP);
}

int i=0;

void loop()
{
    int tipka_je_pritisnjena = !digitalRead(A0);
    if (tipka_je_pritisnjena)
    {
      i++;
      while (tipka_je_pritisnjena)
        tipka_je_pritisnjena = !digitalRead(A0);
      int i_je_liho = i % 2;
      if (i_je_liho)
        digitalWrite(13, HIGH);
      else
        digitalWrite(13, LOW);
    }
}
```

4 KOMUNIKACIJSKI VMESNIKI

4.1 Serijska komunikacija UART

Nekaj več o UART komunikaciji si lahko preberete vsepovsod na svetovnem spletu. Ker jo uporabljamo že več kot pol stoletja, lahko rečemo, da sodi med osnovne komunikacijske protokole.



Slika 4.1: Časovni potek napetosti na kominikacijski povezavi.

4.1.1 NALOGA: Osnovni parametri UART protokola

1. Preučite UART protokol in prerišite časovni potek signala (teoretično).
2. Na teoretičnem primeru komentirajte:

- pomen napetostnega signala,
- kako si sledijo podatkovne informacije,
- označite start in stop bit,
- izračunajte dolžino trajanja enega bita pri $baud = 9600b/s$.

Komunikacija UART je tako razširjena, da jo vključujejo v skoraj vse programabilne elektronske komponente in Arduino NANO ni nobena izjema. Mikrokrmilnik ATmega328p vsebuje enoto za komunikacijo UART in je dostopna na priključkih 0 (Rx) in 1 (Tx). Preko te enote lahko pošiljamo/sprejemamo podatke drugih zunanjih naprav.

4.1.2 NALOGA: Uporaba serijske UART komunikacije

1. Preučite shemo za krmilnik Arduino NANO in poiščite priključka za UART komunikacijo.
2. Preskusite naslednji program mikrokrmilnika za pošiljanje nekega besedila računalniku in odziv spremjajte v serijskemu oknu programa ArduinoIDE:

```
void setup() {
      Serial.begin(9600);
}
void loop() {
  Serial.println("Pozdravljen svet.");
  delay(1000);
}
```

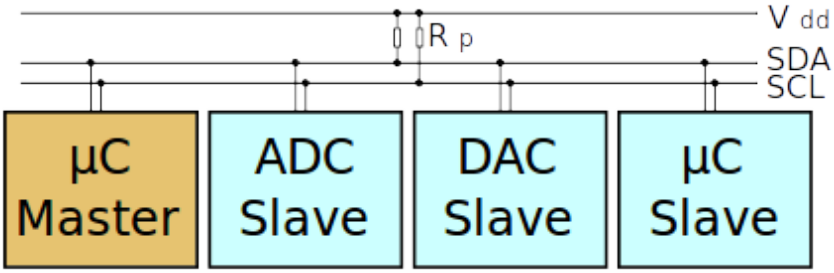
4.1.3 NALOGA: Časovni potek napetosti UART komunikacije

1. Z osciloskopom posnemite napetostni signal pošiljanja enega samega znaka in
2. parametre primerjajte s teoretičnimi vrednostmi komunikacije.
3. Na grafu U(t) označite logične vrednosti posameznih bitov in označite njihovo funkcijo (start bit, stop bit in položaj bita D0..D7).
4. Iz grafa U(t) odčitajte poslano podatkovno vrednost in jo primerjajte z ASCII tabelo.

```
  Serial.print("A");
```

4.2 I2C komunikacija

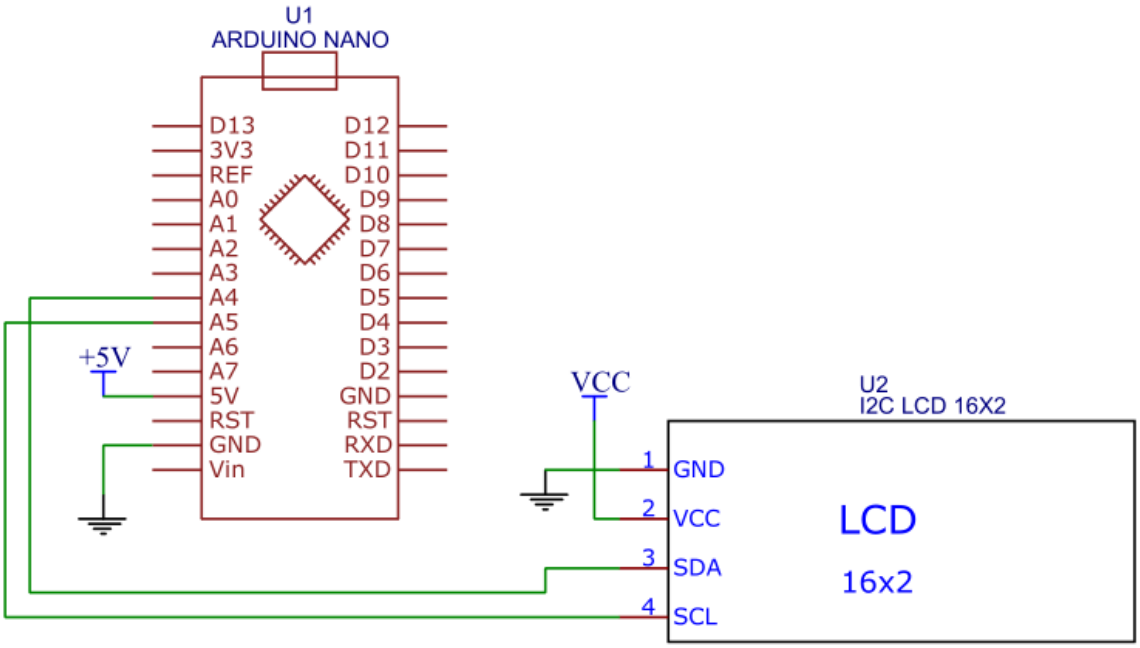
Komunikacija lahko poteka tudi na drugačne načine, na primer med več napravami. Ena takih komunikacij je t.i. I2C komunikacija. Več o tej komunikaciji si lahko preberemo na wikipediji o I2C podatkovnem vodilu



Slika 4.2: Na I2C vodilo priključene naprave.

V primeru, ki ga prikazuje sl. 4.2 je glavna naprava označena kot »master«, ki bo v našem primeru Arduino NANO. Ostale naprave pa so »podložniki«. Vsak od njih mora imeti svoj naslov in mora zanj glavna naprava vedeti, saj le tako lahko vzpostavi komunikacijo z njim (podobno kot IP številke v TCP/IP omrežju).

Naslove podložnikov včasih lahko nastavimo ročno na podložniku ali pa so zapisani že v sami napravi podložnika. Slednjo situacijo si lahko ogledamo na primeru LCD z I2C vodilom.



Slika 4.3: Priključitev LCD-ja na I2C vodilo.

4.2.1 NALOGA: Priključitev I2C LCD-ja

1. Priključite LCD z I2C vodilom na Arduino NANO tako, kot prikazuje sl. 4.3 in s programom, ki ga najdete na Arduino strani, ugotovite njegov naslov, ter ga zapišite : _____

Ker je sam protokol komunikacije bolj zapleten, bomo v ta namen uporabljali knjižnico LiquidCrystal_I2C.h. Knjižnico lahko namestimo v Arduino IDE tako:

```
Sketch -> Include Library -> Manage Libraries

Filter your Serch... : LiquidCrystal I2C (by Marco Schwartz)
```

4.2.2 NALOGA: Izpis na LCD

1. Preskusite naslednji program in
2. nastavite primeren kontrast LCDja.
3. Vezju dodajte še potenciometer, s katerim bomo lahko poljubno nastavljali napetostni potencial in ga merili na priključku A0.
4. Spremenite program tako, da boste izpisovali na LCD izpisovali napetost in ne ADC vrednost.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
int adcValue = 0;
const int POT_PIN = A0;
void setup() {
  pinMode(POT_PIN,INPUT);
  lcd.init();
  lcd.init();
  lcd.backlight();
}
void loop() {
  adcValue = analogRead(POT_PIN);
  lcd.clear();
  lcd.print("potenciometer:");
  lcd.setCursor(0,1);
  lcd.print(adcValue);
  delay(200);
}
```

5 MODULACIJA

5.1 Amplitudna modulacija

Naprimer, da želimo poslati nek podatek v digitalni obliki. Le-ta je seveda sestavljen iz logičnih “1” in “0” (zgoraj na sliki5 5.1). Pri prenosih signalov le-te pogosto moduliramo. Digitalna amplitudna modulacija je ena najpreprostejših modulacij, ki ga prikazuje slika 5.1 na točki 3. Tako moduliran signal se večinoma uporablja za prenos podatkov daljinskega upravljalnika hišnih naprav (TV, setup-box, radio …). Za sprejemanje in demudolacijo teh signalov pa najcečkrat uporabimo elektronski element kot naprimer TSOP32230.

> ### 5.1.1 NALOGA: Pošiljanje IR signala
>
> 1. nosilno frekvenco
> 2. serial (ime priimek)
> 3. modulacija
> 4. IR dioda…

5.2 Frekvenčna modulacija

> ### 5.2.1 NALOGA: Blinker

5.3 Pulzno-širinska modulacija

> ### 5.3.1 NALOGA: Krmiljenje servo-motorja